

Intro:

This is an attempt to interact with deepseek on coding parts of homework 10. The purpose of this study is to better understand how to prompt/interact with LLMs more effectively and LLM's capability of solving real life coding problems with few-shot prompting. The specific model I interacted with was DeepSeek-V3.2. I used Deepseek's web ui to interact with the model. I will focus on how different prompting methods (or modes of reasoning) affect a model's one-shot correctness of the problems, and how to improve its accuracy without providing more in context examples. Note that in this report I will only include screenshots of important parts of the conversation. For full conversation traces, please refer to the conversation links.

Conversation Links:

<https://chat.deepseek.com/share/8vswr9xhltacersq2x>
<https://chat.deepseek.com/share/z3d2wlzyriby9tat3v>
<https://chat.deepseek.com/share/b9ei80b7h70hg32ego>
<https://chat.deepseek.com/share/svy7a295g0hypncpej>

Annotated Traces:

I started the first question by providing context to the LLM. Because I am interacting with the LLM on the web ui, the LLM does not have any information about my local coding environment or requirements, thus it is important to first provide necessary information so that the LLM can answer correctly in an expected way. I started my prompt by telling the LLM this is a deep learning class homework coding assignment, and I am completing a google colab notebook:

You are working on a deep learning class coding assignment. You are completing a google colab notebook:

Below, you'll find a simple transformer implementation in Numpy that we have provided for you. It's important to note that this implementation is different from a Transformer in real applications. The differences include:

Only a single layer with a single head is in the network.

There are no residual connections.

There is no normalization or dropout.

We concatenate the positional encoding rather than adding it to the inputs.

There are no activation functions or MLP layers.

It does not support attention masking.

The input is a single sequence instead of a batch. So there is no need to implement padding.

To ensure that you understand the transformer model fully, your task is to implement a PyTorch equivalent model. You don't need to include the printing and plotting code found in the Numpy version. You should implement a vectorized version of the attention operation, meaning that you should calculate all attention scores at once, rather than looping over keys. Once you have completed your implementation, make sure it passes the tests included in the cell below.

Implement the PytorchTransformer class. It should be identical to the forward pass of the NumpyTransformer class.

Hint: The attention operation should be implemented as:

```
softmax(QKTdk--v)V
```

where the softmax is applied to the last dimension, meaning that the

```
class NumpyTransformer:
    def __init__(self, Km, Qm, Vm, pos=None):
        """
        # Km, Qm, Vm are the matrices that will be used to compute the
        attention
        # Km and Qm are size (input_dim + pos_dim, qk_dim), and Vm is
        (input_dim + pos_dim, v_dim).
        # pos is an array of positional encodings of shape
        (max_seq_len, pos_dim) that will be concatenated to the input
        sequence
        """
        self.Km = Km
        self.Qm = Qm
        self.Vm = Vm
        self.pos = pos
        self.qk_dim = Qm.shape[1]
```

```
    def forward(self, seq, verbose=False, plot=False):
        # seq is a numpy array of shape (seq_len, input_dim). There is
        no batch dimension.
```

```
        # Concatenate positional encodings if they are provided
        if self.pos is not None:
            seq = np.concatenate([seq, self.pos[:seq.shape[0]]], axis=-1)
        K = seq @ self.Km # seq_len x qk_dim
        Q = seq @ self.Qm # seq_len x qk_dim
        V = seq @ self.Vm # seq_len x v_dim
        if verbose:
            print('Keys', K.tolist())
            print('Queries', Q.tolist())
            print('Values', V.tolist())
        if plot:
```

```
class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        #####
        #####
        # TODO: Define query, key, value projection layers Qm, Km, Vm.
        # Each of them is a linear projection without bias

        #####
        #####

        self.d_k = qk_dim

    def forward(self, seq):
        """
        Transformer forward pass

        Inputs: seq is a torch tensor of shape (seq_len, input_dim).
        Outputs: a torch tensor of shape (seq_len, v_dim), the output of the attention
        operation
        """

        #####
        #####
        # TODO: Implement the forward pass of the 'PytorchTransformer' class.
        # The forward pass should be identical to the forward pass of the
```

```
#####
#####
# TODO: Implement the forward pass of the 'PytorchTransformer' class.
# The forward pass should be identical to the forward pass of the
# 'NumpyTransformer' class.
#
# Hint: The attention operation should be implemented as
# If 'pos' exists, it should be concatenated to the input sequence.

#####
#####
#####
#####
# END OF YOUR CODE

#####
#####
return out

test()
```

Note that instead of dumping the whole google colab notebook to the model, I prompt the model to do one subpart at a time, while at the same time manually filtered out information that the model don't necessarily need. This is because this report primarily focuses on investigating ways to more efficiently interact with the model, and dumping the whole

notebook to the model at once makes it hard to isolate causes of incorrectness. However, dumping everything to the model at once seems an interesting direction to investigate—afterall, it will be the most efficient way to do the problems using a LLM.

The LLM gives this:

Looking at the Numpy implementation, I need to create a PyTorch equivalent. Let me analyze what the NumpyTransformer does:

1. Concatenates positional encodings to the input sequence
2. Projects the sequence to Keys, Queries, and Values using linear transformations
3. Computes attention scores using dot products between Q and K
4. Applies softmax after scaling by $\sqrt{\text{qk_dim}}$
5. Multiplies attention weights with Values

Here's the PyTorch implementation:

```
python  

import torch
import torch.nn as nn
import numpy as np

class PytorchTransformer(nn.Module):
    def __init__(self, input_dim, qk_dim, v_dim, pos_dim=None, max_seq_len=10):
        super().__init__()
        if pos_dim is not None:
            self.pos = nn.Embedding(max_seq_len, pos_dim)
        else:
            self.pos = None
        in_dim = input_dim
        if pos_dim is not None:
            in_dim += pos_dim

        #####
        # TODO: Define query, key, value projection layers Qn, Km, Vm.
        # Each of them is a linear projection without bias
        #####
        self.Km = nn.Linear(in_dim, qk_dim, bias=False)
        self.Qm = nn.Linear(in_dim, qk_dim, bias=False)
        self.Vm = nn.Linear(in_dim, v_dim, bias=False)
        #####

        self.d_k = qk_dim
```

```
def forward(self, seq):
    """
    Transformer forward pass

    Inputs: seq is a torch tensor of shape (seq_len, input_dim).
    Outputs: a torch tensor of shape (seq_len, v_dim), the output of the attention operation
    """
    #####
    # TODO: Implement the forward pass of the 'PytorchTransformer' class.
    # The forward pass should be identical to the forward pass of the
    # 'NumpyTransformer' class.
    #
    # Hint: The attention operation should be implemented as
    # If 'pos' exists, it should be concatenated to the input sequence.
    #####

    seq_len = seq.shape[0]

    # Concatenate positional encodings if they are provided
    if self.pos is not None:
        positions = torch.arange(seq_len, device=seq.device)
        pos_emb = self.pos(positions) # (seq_len, pos_dim)
        seq = torch.cat([seq, pos_emb], dim=-1) # (seq_len, input_dim + pos_dim)

    # Project to Keys, Queries, and Values
    K = self.Km(seq) # (seq_len, qk_dim)
    Q = self.Qm(seq) # (seq_len, qk_dim)
    V = self.Vm(seq) # (seq_len, v_dim)

    # Compute attention scores: Q @ K^T / sqrt(d_k)
    # Q: (seq_len, qk_dim), K: (seq_len, qk_dim) -> K^T: (qk_dim, seq_len)
    # Result: (seq_len, seq_len)
    attn_scores = torch.matmul(Q, K.transpose(0, 1)) / torch.sqrt(torch.tensor(self.d_k, dtype=torch.float))

    # Apply softmax along the last dimension (keys dimension)
    attn_weights = torch.softmax(attn_scores, dim=-1) # (seq_len, seq_len)

    # Apply attention weights to values
    out = torch.matmul(attn_weights, V) # (seq_len, v_dim)
    #####
    # END OF YOUR CODE
    #####
    return out
```

Overall, the model was able to immediately understand the assignment and complete the code directly. It understands the assignment, knows where to fill in, and clearly labels the part it added. To evaluate the correctness of the model's answer, I manually examined the code and model's explanation, pasted the added part to google colab and ran the test.

For the next subpart, I prompted the same way (telling model the question prompt, relevant code context, and code with specific comments saying where to fill in the blank with a

TODO comment. The actual prompt can be found in the link to the full conversation. Here I will just present the model answer:

Looking at this identity task, I need to design K_m , Q_m , and V_m matrices that will make the transformer output the same as its input. The key insight is that we want each token to attend to itself.

Let me think through this:

1. **Goal:** For input $[A, B, C, C]$ $\rightarrow$ output $[A, B, C, C]$
2. **Approach:** Make each query attend strongly to its corresponding key
3. **Mechanism:** Use orthogonal keys and queries so each token's query matches best with its own key

Here's my solution:

```
python
# TODO: Write NumPy arrays for "K", "Q", and "V".
# The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and
# (input_dim, v_dim), respectively.
# In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
# choose, but 3 is a reasonable choice.
# For identity operation, we want each position to attend to itself
# Let's use orthogonal vectors for keys and queries
Km = np.eye(3) # Keys: A->[1,0,0], B->[0,1,0], C->[0,0,1]
Qm = np.eye(3) # Queries: same as keys so each token attends to itself
Vm = np.eye(3) # Values: Identity mapping so output equals input
##### END OF YOUR CODE #####
```

Let me verify this works with the helper functions:

```
python
# Test the keys, queries, and values
all_token_seq = np.eye(3) # [A, B, C]
K = all_token_seq @ Km # Should be [[1,0,0], [0,1,0], [0,0,1]]
Q = all_token_seq @ Qm # Should be [[1,0,0], [0,1,0], [0,0,1]]
V = all_token_seq @ Vm # Should be [[1,0,0], [0,1,0], [0,0,1]]

# Test attention patterns
softmax = lambda x: np.exp(x) / np.sum(np.exp(x), axis=-1, keepdims=True)

# For query A = [1,0,0]
attention_A = softmax(Q[0] @ K.T) # Should be [1,0,0] - attends only to A
# For query B = [0,1,0]
attention_B = softmax(Q[1] @ K.T) # Should be [0,1,0] - attends only to B
# For query C = [0,0,1]
attention_C = softmax(Q[2] @ K.T) # Should be [0,0,1] - attends only to C
```

When this is executed:

Why this works:

1. **Keys and Queries:** Both are identity matrices, so:
 - Token A has key = $[1,0,0]$ and query = $[1,0,0]$
 - Token B has key = $[0,1,0]$ and query = $[0,1,0]$
 - Token C has key = $[0,0,1]$ and query = $[0,0,1]$
2. **Attention Calculation:** When we compute $Q @ K^T$:
 - Query A @ Key A = 1 (high score)
 - Query A @ Key B = 0 (low score)
 - Query A @ Key C = 0 (low score)
 - After softmax, Query A attends 100% to Key A
3. **Values:** Identity matrix means:
 - Value A = $[1,0,0]$ (token A)
 - Value B = $[0,1,0]$ (token B)
 - Value C = $[0,0,1]$ (token C)
4. **Output:** Since each query attends only to its corresponding key-value pair, the output equals the input.

For repeated tokens: When there are multiple C tokens, each C query will attend to all C keys equally (since they have identical content), and the weighted average of C values will still be C.

This solution satisfies all the test cases:

- $[A, B, C, C] \rightarrow$ Each position attends to itself: $A \rightarrow A, B \rightarrow B, C \rightarrow C, C \rightarrow C$
- $[C, A, C] \rightarrow C \rightarrow C, A \rightarrow A, C \rightarrow C$
- $[B, B, C] \rightarrow B \rightarrow B, B \rightarrow B, C \rightarrow C$



However, when I plug the answer to the notebook, the current answer is failing the test:

```
FAIL: [[1 0 0]
[0 0 1]] -> [[0.64045748 0. 0.35954252]
[0.35954252 0. 0.64045748]] != [[1 0 0]
[0 0 1]]
FAIL: [[1 0 0]
[0 0 1]] -> [[0.64045748 0. 0.35954252]
[0.35954252 0. 0.64045748]] != [[1 0 0]
[0 0 1]]
FAIL: [[1 0 0]
[0 1 0]] -> [[0.64045748 0.35954252 0.
[0.35954252 0.64045748 0.
]] != [[1 0 0]
[0 1 0]]
FAIL: [[0 0 1]
[0 0 1]
[0 0 1]
[1 0 0]] -> [[0.13616675 0.13616675 0.72766649]
[0.13616675 0.13616675 0.72766649]
[0.13616675 0.13616675 0.72766649]
[0.17297111 0.30811555 0.51891334]
[0.30811555 0.17297111 0.51891334]] != [[0 0 1]
[0 0 1]
[0 0 1]
[1 0 0]]
FAIL: [[1 0 0]
[0 1 0]] -> [[0.64045748 0.35954252 0.
[0.35954252 0.64045748 0.
]] != [[1 0 0]
[0 1 0]]
FAIL: [[0 1 0]
[0 0 1]
[0 1 0]
[0 0 1]] -> [[0. 0.64045748 0.35954252]
[0. 0.35954252 0.64045748]
[0. 0.64045748 0.35954252]
[0. 0.35954252 0.64045748]] != [[0 1 0]
[0 0 1]
[0 1 0]
[0 0 1]]
FAIL: [[0 1 0]
[0 1 0]
[1 0 0]
[0 0 1]
[0 1 0]] -> [[0.13616675 0.72766649 0.13616675]
[0.13616675 0.72766649 0.13616675]
[0.30811555 0.51891334 0.17297111]
[0.17297111 0.51891334 0.30811555]
```

Now I test two methods : 1. Turning on the DeepThink feature (a feature to enable better reasoning from the model). 2. Iterative debugging (give the test feedback to the model and ask it to refine itself).

1. Turning on DeepThink Feature

I started another fresh conversation (to avoid past context affecting the chat), provided the model with the exact same prompt, with DeepThink feature on. However, even with this feature on, the model still output the exact wrong answer:

Let's solve this together.

Looking at this problem, I need to design K_m , Q_m , and V_m matrices that will make the transformer perform the identity operation on sequences of one-hot vectors.

The key insight is that we want each token to attend to itself. Since we have one-hot encoded tokens $[A, B, C] = [[1,0,0], [0,1,0], [0,0,1]]$, we can use orthogonal vectors in the key/query space to ensure each token only attends to itself.

Here's my solution:

```
python Copy Download

# Since we want identity operation and each token should attend to itself,
# we can use orthogonal keys and queries that make each token only attend to itself
# even when there are repeated tokens in the sequence.

Km = np.eye(3) # Identity matrix for keys
Qm = np.eye(3) # Identity matrix for queries
Vm = np.eye(3) # Identity matrix for values
```

Let's try iterative debugging by providing the model with the test feedbacks:

```
FAIL: [[0 1 0]
[1 0 0]] -> [[0.35954252 0.64045748 0.    ]
[0.64045748 0.35954252 0.    ]] != [[0 1 0]
[1 0 0]]
FAIL: [[0 0 1]
[1 0 0]]
[1 0 0]] -> [[0.52891692 0.    0.47108308]
[0.78082789 0.    0.21917211]
[0.78082789 0.    0.21917211]] != [[0 0 1]
[1 0 0]]
FAIL: [[0 1 0]
[1 0 0]] -> [[0.35954252 0.64045748 0.    ]
[0.64045748 0.35954252 0.    ]] != [[0 1 0]
[1 0 0]]
FAIL: [[1 0 0]
[0 1 0]] -> [[0.64045748 0.35954252 0.    ]
[0.35954252 0.64045748 0.    ]] != [[1 0 0]
[0 1 0]]
FAIL: [[1 0 0]
[0 0 1]
[1 0 0]
[0 0 1]]
[1 0 0]] -> [[0.72766649 0.    0.27233351]
[0.4571342 0.    0.5428658 ]
[0.72766649 0.    0.27233351]
[0.4571342 0.    0.5428658 ]
[0.72766649 0.    0.27233351]] != [[1 0 0]
[0 0 1]
[1 0 0]
[0 0 1]]
[1 0 0]]
```

Copy Edit

With this test feedback, the model seem to be able to understand what is wrong:

“With the identity matrices, the dot product for a token with itself is 1, but with other tokens it is 0. However, when there are multiple tokens, the softmax of $[1,0,0]$ (for example) is not $[1,0,0]$ but rather $[e^1/(e^1 + e^0 + e^0), \dots]$ which is approximately $[0.576, 0.212, 0.212]$ for a sequence of three tokens. This does not yield the identity.”

And was able to eventually give a correct answer:

So the complete solution is:

```
python Copy Download  
  
scale_factor = 10 # Can be any sufficiently large number  
Km = scale_factor * np.eye(3)  
Qm = scale_factor * np.eye(3)  
Vm = np.eye(3)
```

This ensures that:

1. Each token attends strongly to itself (and other identical tokens)
2. The attention weights approach 1 for unique tokens and equal distribution for repeated tokens
3. The value matrix preserves the original token representation

The key insight is that we need to overcome the `sqrt(qk_dim)` scaling in the attention mechanism by making the dot products sufficiently large.

2. To ablate, let's see only iterative debugging without DeepThink feature on:

I see the issue! The problem is that when we use identity matrices for both Km and Qm, the dot products between different tokens can be non-zero when they share dimensions. For example:

- Query A = [1,0,0] and Key B = [0,1,0] gives 0 (good)
- But Query A = [1,0,0] and Key A = [1,0,0] gives 1 (good)
- However, when we have multiple tokens, the softmax normalizes across all positions, so even small non-zero scores can cause issues.

The solution is to make the dot product between a token's query and its own key much larger than with other keys. Let me fix this:

```
python Copy Download  
  
#####  
# TODO: Write Numpy arrays for `Km`, `Qm`, and `Vm`.  
# The dimensions should be (input_dim, qk_dim), (input_dim, qk_dim), and  
# (input_dim, v_dim), respectively.  
# In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you  
# choose, but 3 is a reasonable choice.  
#####  
  
# Use larger values to make self-attention dominant  
Km = np.eye(3) * 10 # Scale up to make dot products larger  
Qm = np.eye(3) * 10 # Scale up to make dot products larger  
Vm = np.eye(3)      # Values: identity mapping  
  
##### END OF YOUR CODE #####
```

Let me test why this works:

It seems like the DeepThink feature was not the main reason for the correct answer. It was the test feedback that enabled the model to reflect on its mistakes and refine.

Although the model did get the correct answer, the result was dependent on one round of iterative feedback. Here, I further explore if it is possible to get the model to answer correctly one-shot. In addition to the original prompt, I provided the model with a test

softmax run, as shown in the “hint” cell:

```
... ===== Hint 1 =====
Selecting index 0 [9.99753241e-01 1.23379352e-04 1.23379352e-04]
Selecting index 1 [3.35334911e-04 9.99619282e-01 4.53826452e-05]
===== Hint 2 =====
Selecting index 0 [9.99954602e-01 4.53978687e-05]
Selecting index 1 [0.00669285 0.99330715]
```

However, with or without DeepThink feature on, the model was not able to get the correct answer in one-shot. However, with the DeepThink feature on, something interesting happened: the model was able to get the correct answer in the reasoning part:

So this works.

However, note that the input to the value matrix is the one-hot vector, and V_m is I , so the value is the one-hot vector.

We don't need to scale the value matrix by k , because we want the output to be the one-hot vector.

Therefore, we set:

```
text                                                                    Copy Download

Km = k * I
Qm = k * I
Vm = I
```

with k being a large number, say 10.

But note: the problem says that when we generate plots, we will rescale the range of the weights and outputs to 0-1, so the absolute values of k don't matter as long as they are large enough to make the softmax approximate a hard assignment for the same tokens.

Let's set $k=10$.

We choose $q_k_dim = 3$, and $v_dim = 3$.

So:

```
text                                                                    Copy Download

Km = 10 * np.eye(3)
Qm = 10 * np.eye(3)
Vm = np.eye(3)
```

Let's code accordingly.

However, after reasoning, in the final answer, the model used an inconsistent answer which

is incorrect:

Let's code accordingly.

Looking at this problem, I need to design K_m , Q_m , and V_m matrices that will make the transformer perform the identity operation on one-hot encoded sequences.

The key insight is that for each token to attend to itself, we need the attention mechanism to focus on tokens with the same content. Since we're dealing with one-hot encoded vectors $[A, B, C] = [[1,0,0], [0,1,0], [0,0,1]]$, we can use the identity matrix to make each token attend to itself.

Here's my solution:

```
python Copy Download  
  
# Since we want each token to attend to itself (identity operation),  
# we can use identity matrices that preserve the one-hot structure  
 $K_m = \text{np.eye}(3)$  # (3, 3)  
 $Q_m = \text{np.eye}(3)$  # (3, 3)  
 $V_m = \text{np.eye}(3)$  # (3, 3)
```

Let me explain why this works:

This suggests two things. First, by providing an example run of softmax from the hint section, the model was able to improve its answer (by being correct in the reasoning section – this means it has enough information to derive the correct answer). This reveals an important weakness of the LLM: it lacks the ability to perform actual calculations. Thus, to help it improve its answer, we either need to enable it to run code and calculations (by exposing a calculator or code interpreter to the model), or provide it with some example run results to help it understand the code better. Second, we also learned that for this specific model, sometimes there are inconsistencies between model reasoning trace and final answer. The exact cause for this is out of scope of this report, but remains a very interesting follow up work to investigate.

For the next subpart, I used the same approach, and the model's first attempt was wrong:

Let me design the solution:

python

 Copy  Download

```
A = np.array([1,0,0])
B = np.array([0,1,0])
C = np.array([0,0,1])

tokens = [A, B, C]

#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#     The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#     The shape of Vm is [input_dim + pos_dim, v_dim].
#     The shape of pos is [max_len, pos_dim].
#     In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#     choose, but 1 is a reasonable choice. max_len is the maximum sequence
#     length you will encounter, 4 in this case.
#     pos_dim can be any value you choose, but 4 is a reasonable choice.
#####

max_len = 4
pos_dim = 4
input_dim = 3
v_dim = 3
qk_dim = 1

# Positional encoding: we want position 0 to be unique and all queries to match it
# Let's make position 0 have a distinctive pattern that all queries can recognize
pos = np.zeros((max_len, pos_dim))
# Position 0 gets a special marker [1,0,0,0]
pos[0] = [1, 0, 0, 0]
# Other positions get different patterns that don't match position 0
pos[1] = [0, 1, 0, 0]
pos[2] = [0, 0, 1, 0]
pos[3] = [0, 0, 0, 1]

# Key matrix: Focus only on positional information, ignore token content
# We want Key for position 0 to be high, others to be low
Km = np.zeros((input_dim + pos_dim, qk_dim))
# Set weights for positional dimensions: position 0 should give high output
Km[3:7] = [[10], [0], [0], [0]] # Only the first positional dimension matters

# Query matrix: ALL queries should look for position 0's pattern
Qm = np.zeros((input_dim + pos_dim, qk_dim))
# Set weights to detect position 0's pattern [1,0,0,0]
Qm[3:7] = [[10], [0], [0], [0]] # Match position 0's encoding

# Value matrix: Output should be the token content from position 0
# So we extract the original token dimensions (first 3 dimensions)
Vm = np.zeros((input_dim + pos_dim, v_dim))
Vm[0:3] = np.eye(3) # Copy the token content to output

##### END OF YOUR CODE #####
```

Again, we use iterative debugging where we provide the failed test run to the model:

```
... FAIL: [[0 1 0]
[0 0 1]
[1 0 0]] -> [[3.72007598e-44 1.00000000e+00 3.72007598e-44]
[3.33333333e-01 3.33333333e-01 3.33333333e-01]
[3.33333333e-01 3.33333333e-01 3.33333333e-01]] != [[0 1 0]
[0 1 0]
[0 1 0]]
FAIL: [[0 0 1]
[0 1 0]] -> [[0.00000000e+00 3.72007598e-44 1.00000000e+00]
[0.00000000e+00 5.00000000e-01 5.00000000e-01]] != [[0 0 1]
[0 0 1]]
FAIL: [[0 0 1]
[0 0 1]
[1 0 0]] -> [[3.72007598e-44 0.00000000e+00 1.00000000e+00]
[3.33333333e-01 0.00000000e+00 6.66666667e-01]
[3.33333333e-01 0.00000000e+00 6.66666667e-01]] != [[0 0 1]
[0 0 1]
[0 0 1]]
FAIL: [[1 0 0]
[0 0 1]
[1 0 0]] -> [[1.00000000e+00 0.00000000e+00 3.72007598e-44]
[6.66666667e-01 0.00000000e+00 3.33333333e-01]
[6.66666667e-01 0.00000000e+00 3.33333333e-01]] != [[1 0 0]
[1 0 0]
[1 0 0]]
FAIL: [[0 1 0]
[0 0 1]
[0 1 0]] -> [[0.00000000e+00 1.00000000e+00 3.72007598e-44]
[0.00000000e+00 7.50000000e-01 2.50000000e-01]
[0.00000000e+00 7.50000000e-01 2.50000000e-01]
[0.00000000e+00 7.50000000e-01 2.50000000e-01]] != [[0 1 0]
[0 1 0]
[0 1 0]
[0 1 0]]
FAIL: [[0 0 1]
[0 1 0]
[0 0 1]
[1 0 0]] -> [[3.72007598e-44 3.72007598e-44 1.00000000e+00]
[2.50000000e-01 2.50000000e-01 5.00000000e-01]
[2.50000000e-01 2.50000000e-01 5.00000000e-01]
[2.50000000e-01 2.50000000e-01 5.00000000e-01]] != [[0 0 1]
[0 0 1]
[0 0 1]
[0 0 1]]
```

And the model was able to generate correct result:

I see the issue! The problem is that all queries are producing the same output (10), which means all positions have equal attention scores. I need to make the keys more distinctive so that only position 0's key gives a high score when multiplied with any query.

Let me fix this by making the keys and queries work together better:

```
python Copy Download

#####
# TODO: Implement numpy arrays for Km, Qm, and Vm and pos.
#   The shape of Km, and Qm are [input_dim + pos_dim, qk_dim].
#   The shape of Vm is [input_dim + pos_dim, v_dim].
#   The shape of pos is [max_len, pos_dim].
#   In this case, input_dim = 3, and v_dim = 3. qk_dim can be any value you
#   choose, but 1 is a reasonable choice. max_len is the maximum sequence
#   length you will encounter, 4 in this case.
#   pos_dim can be any value you choose, but 4 is a reasonable choice.
#####

max_len = 4
pos_dim = 4
input_dim = 3
v_dim = 3
qk_dim = 1

# Positional encoding: Make position 0 unique and all other positions the same
pos = np.zeros((max_len, pos_dim))
# Position 0 gets a special marker [1,0,0,0]
pos[0] = [1, 0, 0, 0]
# All other positions get the same pattern [0,1,0,0] so they're indistinguishable
for i in range(1, max_len):
    pos[i] = [0, 1, 0, 0]

# Key matrix: Position 0 should give high output, others should give low output
Km = np.zeros((input_dim + pos_dim, qk_dim))
# Set weights: position 0 pattern [1,0,0,0] gives high output, [0,1,0,0] gives low
Km[3:7] = [[10], [-10], [0], [0]] # [1,0,0,0] -> 10, [0,1,0,0] -> -10

# Query matrix: ALL queries should give high output regardless of position
Qm = np.zeros((input_dim + pos_dim, qk_dim))
# Set weights to give high output for both position patterns
Qm[3:7] = [[1], [1], [0], [0]] # Both [1,0,0,0] and [0,1,0,0] give positive output

# Value matrix: Output should be the token content from whatever position we attend to
# But we want to output the token from position 0, so we extract original token dimensions
Vm = np.zeros((input_dim + pos_dim, v_dim))
Vm[0:3] = np.eye(3) # Copy the token content to output

##### END OF YOUR CODE #####
```

Here, the lesson is the same with the last subpart.

Q_summerize_part 1:

For this question, my approach to prompt the model is the same. Tell the model the context is that this is a homework coding assignment in google colab for a deep learning course. In addition, because the scope of this question is somewhat more broad, to avoid model confusion, I executed every cell before the “TODO” cell and gave the model the execution

results, for example, a part of my prompt:

We'll use the XSum dataset (<https://arxiv.org/abs/1808.08745>) to train and evaluate our Transformer model. Each example in the XSum dataset consists of a document and its corresponding summary. Below are the dataset statistics and a sample example:

```
DatasetDict({
  train: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 204045
  })
  validation: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 11332
  })
  test: Dataset({
    features: ['document', 'summary', 'id'],
    num_rows: 11334
  })
})
```

Due to the original XSum dataset's large size, we'll use a subset for training and

This should be helpful to the model the same way it is useful to humans – giving more information about the format of dataset, setup, etc.

Here is the model's answer:

Looking at this implementation, I need to create a scaled dot-product attention function that handles both padding masks and causal masks. Let me implement this step by step:

```
python Copy Download

def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    """
    # TODO: Implement this function
    """

    # Get dimensions
    n, tgt_len, d_head = q.shape
    _, src_len, _ = k.shape

    # Compute scaled dot-product attention scores
    # Q: [n, tgt_len, d_head], K: [n, src_len, d_head] -> scores: [n, tgt_len, src_len]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_head)

    # Apply causal mask if needed
    if causal:
        # Create causal mask - Lower triangular matrix
        # Future positions (j > i) should be masked
        causal_mask = torch.triu(torch.ones(tgt_len, src_len) * float('-inf'), diagonal=1)
        causal_mask = causal_mask.to(scores.device)
        scores = scores + causal_mask

    # Apply key padding mask if provided
    if key_padding_mask is not None:
        # key_padding_mask: [n, src_len] with 1s for padding tokens
        # We need to expand it to [n, tgt_len, src_len]
        padding_mask = key_padding_mask.unsqueeze(1).expand(n, tgt_len, src_len)
        # Set padding positions to -inf
        scores = scores.masked_fill(padding_mask, float('-inf'))

    # Apply softmax to get attention weights
    attn_weights = torch.softmax(scores, dim=-1)

    # Apply attention weights to values
    # attn_weights: [n, tgt_len, src_len], V: [n, src_len, d_head] -> output: [n, tgt_len, d_head]
    output = torch.matmul(attn_weights, v)

    return output
```

Let me break down the implementation:

The model answer ran into a runtime error that the “masked_fill” function model expected the second mask to be booleans, but they are actually byte tensors. Although the model was able to fix this after one round of iterative debugging:

```
-----
RuntimeError                                Traceback (most recent call last)
/tmp/ipython-input-358993808.py in <cell line: 0>()
    65 TO_SAVE["scaled_dot_product_attention.6.value"] = o6.view(-1)[3:
    8].tolist()
    66
--> 67 test_scaled_dot_product_attention()

1 frames
/tmp/ipython-input-358993808.py in test_scaled_dot_product_attention()
    32
    33 key_padding_mask = torch.tensor([[0, 0, 1]]).byte()
--> 34 o4 = scaled_dot_product_attention(q1, k1, v1,
key_padding_mask=key_padding_mask)
    35 assert list(o4.shape) == [1, 2, 3]
    36 assert torch.allclose(

/tmp/ipython-input-1128775360.py in scaled_dot_product_attention(q, k, v,
key_padding_mask, causal)
    27 padding_mask = key_padding_mask.unsqueeze(1).expand(n, tgt_len,
src_len)
    28 # Set padding positions to -inf
--> 29 scores = scores.masked_fill(padding_mask, float('-inf'))
    30
    31 # Apply softmax to get attention weights

RuntimeError: masked_fill_ only supports boolean masks, but got mask with dtype
unsigned char
```



The error indicates that the mask should be boolean, not byte. Let me fix this by converting the mask to boolean:

```
python
Copy Download

def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    """
    # TODO: implement this function
    """

    # Get dimensions
    n, tgt_len, d_head = q.shape
    _, src_len, _ = k.shape

    # Compute scaled dot-product attention scores
    # Q: [n, tgt_len, d_head], K: [n, src_len, d_head] -> scores: [n, tgt_len, src_len]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_head)

    # Apply causal mask if needed
    if causal:
        # Create causal mask - lower triangular matrix
        # Future positions (j > i) should be masked
        causal_mask = torch.triu(torch.ones(tgt_len, src_len) * float('-inf'), diagonal=1)
        causal_mask = causal_mask.to(scores.device)
        scores = scores + causal_mask

    # Apply key padding mask if provided
    if key_padding_mask is not None:
        # key_padding_mask: [n, src_len] with 1s for padding tokens
        # Convert to boolean mask
        padding_mask = key_padding_mask.bool()
        # We need to expand it to [n, tgt_len, src_len]
        padding_mask = padding_mask.unsqueeze(1).expand(n, tgt_len, src_len)
        # Set padding positions to -inf
        scores = scores.masked_fill(padding_mask, float('-inf'))

    # Apply softmax to get attention weights
    attn_weights = torch.softmax(scores, dim=-1)

    # Apply attention weights to values
    # attn_weights: [n, tgt_len, src_len], V: [n, src_len, d_head] -> output: [n, tgt_len, d_head]
    output = torch.matmul(attn_weights, v)

    return output
    """
```

The result is still not satisfactory because in the original prompt, it was clearly mentioned that `key_padding_mask` is a byte tensor set to 1 in pad token positions.

However, consider two essential details: padding and causal masking.

padding: The Transformer input usually contains sentences of varying lengths, padded into a tensor. Attention should ignore pad tokens, so they don't impact the results. `key_padding_mask` is a byte tensor set to 1 in pad token positions. If `key_padding_mask` is None, there's no padding in the input.

causal masking: Autoregressive generation in the decoder uses causal attention masks, meaning position i can only attend to position j if $i \geq j$. If causal is set to true, apply causal attention masking. The provided `future_mask` may be useful. Implement scaled dot-product attention below. Ensure your implementation is fully vectorized (no for-loops allowed). Avoid using `torch.nn.functional.multi_head_attention_forward` or any classes in `torch.nn`.

Hint: Consider using `einops` (<https://einops.rocks/>).

As a result, a strong model should have sufficient information to solve the problem in one-shot. Let's see how the model perform one-shot with the same prompt but DeepThink feature on:

Thought for 76 seconds

Let's implement accordingly.

I'll implement the scaled dot-product attention function step by step. This is a core component of the Transformer architecture.

```
python
Copy Download

def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    """
    Scaled Dot-Product Attention

    Args:
        q: Tensor[n, tgt_len, d_head]
        k: Tensor[n, src_len, d_head]
        v: Tensor[n, src_len, d_head]
        key_padding_mask: Tensor[n, src_len] (1 for padding tokens, 0 for real tokens)
        causal: bool (whether to apply causal masking for decoder)

    Returns:
        output: Tensor[n, tgt_len, d_head]
    """
    n, tgt_len, d_head = q.shape
    src_len = k.shape[1]

    # Step 1: Compute attention scores
    # q: [n, tgt_len, d_head], k: [n, src_len, d_head] -> scores: [n, tgt_len, src_len]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_head)

    # Step 2: Apply masks
    if key_padding_mask is not None:
        # Convert padding mask to attention mask format
        # key_padding_mask: [n, src_len] -> attn_mask: [n, tgt_len, src_len]
        attn_mask = key_padding_mask.unsqueeze(1).expand(n, tgt_len, src_len)
        # Set padding positions to -inf
        scores = scores.masked_fill(attn_mask, float('-inf'))

    if causal:
        # Apply causal mask to prevent attending to future tokens
        causal_mask = torch.triu(torch.ones(tgt_len, src_len, device=q.device) * float('-inf'), diagonal=1)
        scores = scores + causal_mask

    # Step 3: Apply softmax to get attention weights
    attn_weights = F.softmax(scores, dim=-1) # [n, tgt_len, src_len]

    # Step 4: Apply attention weights to values
    output = torch.matmul(attn_weights, v) # [n, tgt_len, d_head]

    return output
```

Turns out the model still wasn't able to avoid the mistake even with stronger reasoning abilities. My first hypothesis was that the prompt was too long, so the model has trouble identifying the correct format of the byte tensor. As a result, I emphasized this with a new prompt, ensuring the model will notice the format of the byte tensor mask:



key_padding_mask is a byte tensor set to 1 in pad token positions. If key_padding_mask is None, there's no padding in the input. Redo the question



```
python Copy Download

def scaled_dot_product_attention(q, k, v, key_padding_mask=None, causal=False):
    """
    Scaled Dot-Product Attention

    Args:
        q: Tensor[n, tgt_len, d_head]
        k: Tensor[n, src_len, d_head]
        v: Tensor[n, src_len, d_head]
        key_padding_mask: Tensor[n, src_len] (1 for padding tokens, 0 for real tokens)
        causal: bool (whether to apply causal masking for decoder)

    Returns:
        output: Tensor[n, tgt_len, d_head]
    """
    n, tgt_len, d_head = q.shape
    src_len = k.shape[1]

    # Step 1: Compute attention scores
    # q: [n, tgt_len, d_head], k: [n, src_len, d_head] -> scores: [n, tgt_len, src_len]
    scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_head)

    # Step 2: Apply masks
    if key_padding_mask is not None:
        # key_padding_mask is byte tensor with 1 for padding tokens
        # Expand to match scores shape: [n, src_len] -> [n, tgt_len, src_len]
        padding_mask = key_padding_mask.unsqueeze(1).expand(-1, tgt_len, -1)
        # Set padding positions to -inf (where mask == 1)
        scores = scores.masked_fill(padding_mask, float('-inf'))

    if causal:
        # Create causal mask: upper triangular matrix with -inf above diagonal
        # future_mask from the question: torch.triu(torch.zeros([1024, 1024]).fill_(float("-inf")),
        1)
        causal_mask = torch.triu(torch.ones(tgt_len, src_len, device=q.device) * float('-inf'), diagonal=1)
        scores = scores + causal_mask.unsqueeze(0) # Add batch dimension

    # Step 3: Apply softmax to get attention weights
    attn_weights = F.softmax(scores, dim=-1) # [n, tgt_len, src_len]

    # Step 4: Apply attention weights to values
    output = torch.matmul(attn_weights, v) # [n, tgt_len, d_head]

    return output
```

Key corrections:

1. **Byte Tensor Handling:** The `key_padding_mask` is a byte tensor where `1` indicates padding tokens that should be masked. I use `masked_fill` directly with this mask, setting those positions to `-inf`.
2. **Causal Mask Construction:** I create the causal mask exactly as shown in the question using `torch.triu` with `diagonal=1` to mask future positions while keeping the current position unmasked.

Now the model realizes the format of the byte tensor, but it still can not avoid the mistake of using `masked_fill` directly on it before converting to boolean. As a result, it can be concluded that the reason for the mistake is not because model can not attend to this small detail, but

rather because some other reasons: maybe the model is not familiar enough with the `torch.masked_fill` method, or maybe the model just doesn't have strong enough reasoning ability to recognize the potential issue. Nevertheless, the fact that the model can easily solve the problem with only one round of iterative debugging showcases how important a runnable, testable environment is for a model to correctly solve a complex problem.

The result for the next subpart is exactly the same: the model was able to get the correct answer after at most one round of iterative debugging:

```
AttributeError                                Traceback (most recent call last)
/tmp/python-input-3929001717.py in <cell line: 0>()
    77     TO_SAVE["multihead_attention.4.1.value"] = o41.view(-1)[126: 131].tolist()
    78
--> 79 test_multihead_attention()

4 frames
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
__getattr__(self, name)
    1962         if name in modules:
    1963             return modules[name]
-> 1964         raise AttributeError(
    1965             f'"{type(self).__name__}" object has no attribute "{name}"'
    1966         )

AttributeError: 'MultiheadAttention' object has no attribute 'd_model'
```

I see the issue. The error is occurring because the test code is looking for a `d_model` attribute, but in our implementation we only have `n_heads` and `d_head`. Let me fix this by adding the `d_model` attribute:

```
python
Copy Download

class MultiheadAttention(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        self.d_model = d_model # Add this line
        self.n_heads = n_heads
        self.d_head = d_model // n_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.o_proj = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, key_padding_mask=None, causal=False):
        # TODO: implement this method

        bsz, tgt_len, _ = q.shape
        bsz, src_len, _ = k.shape

        # Project queries, keys, values
        q_proj = self.q_proj(q) # [bsz, tgt_len, d_model]
        k_proj = self.k_proj(k) # [bsz, src_len, d_model]
        v_proj = self.v_proj(v) # [bsz, src_len, d_model]
```

Here's the implementation of the `make_positions` method and the complete Transformer class:

```
python Copy Download

class Transformer(nn.Module):
    def __init__(self, n_words, pad_id, eos_id, max_len, n_layers, d_model, n_heads, d_ffn, p_drop):
        super().__init__()
        self.pad_id = pad_id
        self.eos_id = eos_id
        self.emb_word = nn.Embedding(n_words, d_model)
        self.emb_pos = nn.Embedding(max_len, d_model)
        self.emb_word.weight.data.uniform_(-0.05, 0.05)
        self.emb_pos.weight.data.uniform_(-0.05, 0.05)
        self.emb_ln = nn.LayerNorm(d_model)
        self.encoder_layers = nn.ModuleList([
            TransformerLayer(False, d_model, n_heads, d_ffn, p_drop)
            for _ in range(n_layers)
        ])
        self.decoder_layers = nn.ModuleList([
            TransformerLayer(True, d_model, n_heads, d_ffn, p_drop)
            for _ in range(n_layers)
        ])
        self.lm_head = nn.Linear(d_model, n_words)
        self.lm_head.weight = self.emb_word.weight # Weight sharing
        self.criterion = nn.CrossEntropyLoss(ignore_index=-100)

    def make_positions(self, input_ids, padding_mask):
        #####
        # TODO: implement this method
        #####
        batch_size, seq_len = input_ids.shape
        # Create position indices [0, 1, 2, ..., seq_len-1] for each sequence in the batch
        positions = torch.arange(seq_len, device=input_ids.device).unsqueeze(0)
        # Expand to batch size: [[0, 1, 2, ...], [0, 1, 2, ...], ...]
        positions = positions.expand(batch_size, seq_len)
        return positions
        #####

    def forward(self, input_ids, attention_mask, labels):
        # Encoder forward pass
        enc_padding_mask = input_ids.eq(self.pad_id).byte()
        enc_pos = self.make_positions(input_ids, enc_padding_mask)
        enc_state = self.emb_ln(self.emb_word(input_ids) + self.emb_pos(enc_pos))
        for layer in self.encoder_layers:
            enc_state = layer(enc_state, enc_padding_mask)

        # Prepare decoder inputs
        decoder_input_ids = labels.new_zeros(labels.shape)
        decoder_input_ids[:, 1:] = labels[:, :-1].clone() # Shift right
        decoder_input_ids[:, 0] = self.eos_id # Start with EOS token
        decoder_input_ids.masked_fill_(decoder_input_ids == -100, self.pad_id) # Replace -100 with
        pad_id
```

Note that this problem asks for a vanilla implementation of the classic transformer structure of the paper “Attention is All You Need”. Such implementation is abundant in the model’s training data and thus it will be fairly easy for the model to implement the transformer given enough information.

For the last coding question, I used the exact same approach as the previous problems. Note that for this problem, I did not include the context of the paper this implementation followed. However, I did provide the model with [architectures.py](#), which includes the helper architecture and classes needed to do the assignment. The model was able to correctly do all questions in one-shot. As this is a relatively easy coding question (most of the TODOs

are only a few lines), I will not include the actual screenshot here. Please refer to the full conversation traces for complete conversation.

Summary:

Overall, the model (DeepSeek-V3.2) showcases quite strong coding abilities and was able to complete almost all coding questions with at most one round of iterative debugging. One round of iterative debugging refers to giving feedback of the executed test result back to the model only once and asking it to refine its own code. For this specific problem set, the most common reason for a model needing iterative debugging was that the model lacked the reasoning ability to predict what exactly would happen when the code it wrote is actually executed. For example, in one of the questions, even though the prompt explicitly says `mask` is a byte array, and the model is well aware of this, it still failed to predict that this will be an incompatible format for a given torch method `mask_fill`, which accepts boolean arrays. These can be potentially solved by enabling better reasoning abilities, while I argue that in this case, a more efficient solution would be “test-time scaling”. Instead of spending numerous computations in improving the model's reasoning ability, one thing we can do to tackle this problem is to simply use another model call with the test feedback. In this specific problem set, this approach has been proven working every time, and simply calling the model one more time uses way less computation than trying to improve the model's native reasoning ability. Taken together, these observations suggest that, at least for this kind of coding workload, investing in smarter test-time strategies like iterative debugging and feedback-driven refinement may be a more practical and cost-effective way to boost reliability than solely focusing on training ever-stronger base models.